

Kubernetes Hands-On

Complete Learning Roadmap

From Zero → Production-Ready Engineer

5 Phases

40+ Topics

Production Tips

This roadmap is designed to take you from **absolute beginner** to a **production-ready Kubernetes engineer**. Follow the phases in order — each builds on the previous. Hands-on practice is highlighted throughout so you gain real muscle memory before you ever touch a live cluster.

PHASE 1 | Foundations

Containers, Docker & Core Concepts

1.1 Why Kubernetes?

■ Containers vs VMs	Understand isolation, images, layers, and why containers changed DevOps
■ The Scaling Problem	Manual vs auto-scaling, downtime during deployments, config drift
■ What K8s Solves	Orchestration, self-healing, service discovery, rolling updates
■ The K8s Ecosystem	CNCF landscape, Helm, Istio, Prometheus, ArgoCD — birds-eye view

1.2 Docker Essentials (prerequisite)

■ Build & Run Containers	Dockerfile, docker build, docker run, port mapping
■ Images & Registries	Docker Hub, ECR, GCR — push/pull images
■ Docker Compose	Multi-container local dev, networking, volumes
■ Cleanup & Best Practices	Multi-stage builds, .dockerignore, minimal base images

1.3 Kubernetes Architecture

■ Control Plane	API Server, etcd, Scheduler, Controller Manager
■ Worker Nodes	kubelet, kube-proxy, container runtime (containerd)
■ kubectl CLI	Your primary tool — config, contexts, namespaces

■ Namespaces

Isolation boundaries, default vs custom, resource scoping



Hands-On — Get Your First Cluster Running

```
# Install minikube + kubectl
```

```
minikube start --driver=docker
```

```
kubectl cluster-info
```

```
kubectl get nodes
```

```
kubectl run nginx --image=nginx --port=80
```

```
kubectl get pods -o wide
```

PRO TIP

Use 'kind' (Kubernetes IN Docker) for multi-node local clusters — it's faster than minikube and closer to real production topology.

2.1 Pods — The Atomic Unit

■ Pod Anatomy	Containers, init containers, sidecars, shared volumes
■ YAML Manifests	apiVersion, kind, metadata, spec — master the structure
■ Pod Lifecycle	Pending → Running → Succeeded/Failed/Unknown
■ Liveness & Readiness	httpGet, exec, tcpSocket probes — critical for production
■ Resource Requests/Limits	CPU & Memory — always set these or get OOMKilled

2.2 Controllers

■ Deployment	ReplicaSet management, rollouts, rollbacks, strategy (RollingUpdate)
■ StatefulSet	Ordered pod naming, stable network IDs — for databases
■ DaemonSet	One pod per node — logging agents, monitoring, CNI plugins
■ Job & CronJob	Batch workloads, completion tracking, concurrency policies

2.3 Services & Networking

■ ClusterIP	Default — internal cluster DNS: ..svc.cluster.local
■ NodePort	Expose on node IP — useful for testing, not for production
■ LoadBalancer	Cloud provider LB provisioning — production ingress entry point
■ Endpoints	How Services find Pods — label selectors, EndpointSlices
■ Ingress	HTTP/HTTPS routing, TLS termination, path-based routing rules

2.4 Configuration & Secrets

■ ConfigMap	Non-sensitive config — env vars, mounted files, app settings
■ Secret	Base64 encoded (not encrypted!) — use Sealed Secrets or Vault in prod
■ Volumes	emptyDir, hostPath, PVC, configMap, secret mount types
■ PV & PVC	PersistentVolume, PersistentVolumeClaim, StorageClass

Hands-On — Deploy a Real App

```
kubectl apply -f deployment.yaml
```

```
kubectl rollout status deployment/my-app
```

```
kubectl set image deployment/my-app app=my-app:v2
```

```
kubectl rollout undo deployment/my-app
```

```
kubectl scale deployment/my-app --replicas=5
```

```
kubectl get svc,ep,ingress -n my-namespace
```

PRO TIP

Always use 'kubectl diff -f manifest.yaml' before applying changes in production — it shows exactly what will change, like a git diff.

PHASE 3 | Observability & Debugging

Logs, Metrics, Tracing & Troubleshooting

3.1 Debugging Like a Pro

■ kubectl describe	Events section is GOLD — always check this first for failures
■ kubectl logs	Previous container logs (--previous), multi-container (-c)
■ kubectl exec	Shell into running pods, run diagnostic commands live
■ kubectl port-forward	Access services locally without Ingress — great for debugging
■ kubectl get events	Cluster-wide event stream — sorted by time for incident response

3.2 Metrics & Monitoring Stack

■ Metrics Server	kubectl top pods/nodes — CPU & memory real-time
■ Prometheus	Scrape metrics, PromQL queries, alerting rules
■ Grafana	Dashboards — Node Exporter, K8s mixin, custom boards
■ Alertmanager	Route alerts to Slack, PagerDuty, email — define SLOs
■ Loki + Promtail	Log aggregation without Elasticsearch overhead

3.3 Common Production Issues

■ CrashLoopBackOff	Check logs --previous, probes misconfigured, OOMKilled
■ Pending Pods	Insufficient resources, taint/toleration mismatch, PVC unbound
■ ImagePullBackOff	Wrong image name/tag, registry auth, private repo credentials
■ Service Unreachable	Label selector mismatch, NetworkPolicy blocking, DNS issues
■ OOMKilled	Memory limit too low — check VPA recommendations

Debug Command Toolkit

```
kubectl describe pod -n
kubectl logs -c --previous --tail=100
kubectl exec -it -- /bin/sh
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl top pods --sort-by=memory -n production
kubectl debug node/ -it --image=ubuntu # node-level debug
```

PRO TIP

Install 'k9s' — a terminal UI for Kubernetes. It transforms debugging speed dramatically. Navigate pods, logs, events with keyboard shortcuts.

PHASE 4 | Security & Production Hardening

RBAC, Policies, Networking & Secrets

4.1 RBAC — Role-Based Access Control

■ ServiceAccounts	Pod identity — always use dedicated SAs, never default
■ Role & ClusterRole	Rules: verbs (get, list, watch, create) on resources
■ RoleBinding	Bind roles to users, groups, or service accounts
■ OIDC Integration	AWS SSO, Google Workspace, Active Directory — real-world auth
■ kubectl auth can-i	Test permissions: 'kubectl auth can-i create pods --as=dev'

4.2 Pod Security

■ Pod Security Standards ■	Privileged / Baseline / Restricted — enforce at namespace level
■ Security Context	runAsNonRoot, readOnlyRootFilesystem, drop ALL capabilities
■ Network Policies	Default deny all, allow only required flows — zero trust
■ Secrets Management	External Secrets Operator, Vault Agent, Sealed Secrets
■ Image Scanning	Trivy, Snyk — scan in CI, never ship vulnerable images

4.3 Resource Management at Scale

■ LimitRange ■	Default requests/limits per namespace — prevents resource starvation
■ ResourceQuota	Cap total CPU/memory/object count per namespace
■ HPA	Horizontal Pod Autoscaler — CPU, memory, custom metrics
■ VPA	Vertical Pod Autoscaler — right-size your resource requests
■ Cluster Autoscaler ■	Auto provision/deprovision nodes — cloud cost optimization

Security Hardening Commands

```
# Apply network policy (default deny all ingress)
kubectl apply -f default-deny-ingress.yaml

# Check RBAC permissions for a service account
kubectl auth can-i --list --as=system:serviceaccount:prod:my-sa

# Scan image for CVEs with Trivy
```

```
trivy image my-app:latest --severity HIGH,CRITICAL
```

```
# Audit your cluster against CIS benchmarks
```

```
kube-bench run --targets master,node
```

PRO TIP

Run 'kubectrl get podsecuritypolicies' or check Pod Security Admission — 90% of breaches exploit privileged containers. Always run as non-root.

PHASE 5 | Production Ops & GitOps

Helm, CI/CD, Cluster Management & SRE

5.1 Helm — Kubernetes Package Manager

■ Helm Charts	Templates, values.yaml, helpers — packaging your apps
■ Chart Repositories	ArtifactHub — find community charts for Prometheus, Nginx, etc.
■ Helm Lifecycle	helm install, upgrade, rollback, uninstall — versioned releases
■ Helm Hooks	pre-install, post-upgrade hooks — DB migrations, tests
■ Helm Test	Validate deployments — 'helm test my-release'

5.2 GitOps with ArgoCD

■ GitOps Principles	Git as single source of truth — declarative, versioned, auditable
■ ArgoCD	Sync K8s state from Git — automatic or manual — drift detection
■ App of Apps	Manage multiple environments from one ArgoCD application tree
■ Notifications	Sync status to Slack, Teams — know when deploys succeed/fail
■ Flux CD	Alternative GitOps tool — tighter Git integration, CNCF graduated

5.3 Multi-Cluster & Advanced Patterns

■ Multi-Cluster	kubectx/kubens for context switching — federated clusters
■ Service Mesh	Istio / Linkerd — mTLS, traffic splitting, circuit breaking
■ Blue/Green Deploy	Zero-downtime — run two environments, switch traffic instantly
■ Canary Releases	Progressive delivery — route 5% traffic, monitor, then 100%
■ Disaster Recovery	Velero backups — etcd snapshots, PV backups, restore drills

5.4 Managed Kubernetes (Cloud)

■ AWS EKS	eksctl, managed node groups, Fargate profiles, IAM roles for SA
■ Azure AKS	AAD integration, Azure CNI, managed control plane, ACR pull
■ GCP GKE	Autopilot mode, Workload Identity, GKE release channels
■ On-Premise	kubeadm, k3s (edge/IoT), Rancher — bare metal considerations

Production Ops Commands

```
helm upgrade --install my-app ./chart -f values.prod.yaml --atomic
```

```
argocd app sync my-app --prune
```

```
kubectrl drain node-1 --ignore-daemonsets --delete-emptydir-data
```

```
kubectrl cordon node-2 # Prevent new pods from scheduling
```

```
velero backup create prod-backup --include-namespaces=production
```

```
kubectrl rollout restart deployment/my-app # Zero-downtime restart
```

PRO TIP

Use 'kubectrl drain' before node maintenance — never delete pods directly in production. It respects PodDisruptionBudgets and ensures HA.

Essential Tools & Certifications

Tool	Purpose	Install
kubectl	Primary CLI — everything goes through it	Official docs
k9s	Terminal UI — game changer for daily ops	k9scli.io
Helm	Package & release management	helm.sh
kubectx/kubens	Fast context & namespace switching	GitHub: ahmetb
stern	Multi-pod log tailing	GitHub: stern
Lens IDE	Desktop GUI for cluster management	k8slens.dev
kind	Local multi-node clusters in Docker	kind.sigs.k8s.io
Trivy	Container vulnerability scanning	aquasecurity.io
ArgoCD	GitOps continuous delivery	argo-cd.readthedocs.io

Certifications to Target

Certification	Level	Focus
CKA	Intermediate	Cluster Administration — most employer-recognized
CKAD	Intermediate	Application Developer — deployments, config, debugging
CKS	Advanced	Security Specialist — requires CKA first
KCNA	Entry	Cloud Native Associate — concepts & ecosystem

FINAL ADVICE

Build real projects: deploy a full-stack app (frontend + backend + DB + Redis) on a local kind cluster. Add Prometheus monitoring, Ingress, secrets, HPA, and ArgoCD. This single project will teach you more than 100 tutorials. Then break it on purpose and fix it — that is how production engineers are made.